

Dossier de remise - MyWorkoutApp

1. Rappel du sujet

Nom du projet : MyWorkoutApp

MyWorkoutApp est une application mobile développée en React Native avec Expo. Le but du projet est de proposer un suivi d'entraînement simple et rapide pendant une séance de musculation, avec un stockage local des données.

Le sujet demandé porte sur cinq fonctionnalités principales :

1. créer une séance avec une date, une durée et un commentaire ;
2. ajouter des exercices dans la séance ;
3. ajouter des séries par exercice avec répétitions et charge ;
4. consulter l'historique des séances et le détail d'une séance ;
5. gérer un objectif hebdomadaire avec progression.

L'application a été pensée pour rester simple, claire et cohérente avec les notions vues au cours.

2. Spécifications des choix

Les fonctionnalités réellement présentes dans l'application sont les suivantes :

- création d'une séance avec date, durée et commentaire ;
- ajout dynamique de plusieurs exercices à une séance ;
- ajout de séries avec poids et nombre de répétitions ;
- marquage d'une série comme complétée ;
- consultation d'un historique des séances ;
- consultation du détail complet d'une séance ;
- affichage d'un objectif hebdomadaire fixé à trois séances avec progression ;
- affichage d'informations simples sur les séances : nombre d'exercices, nombre de séries, séries complétées, volume total et volume moyen ;
- validation des formulaires avec React Hook Form et Zod ;
- stockage local des données dans SQLite ;
- retour haptique lors de la complétion d'une série ;
- gestion des états de chargement, d'erreur, de réessai et de rafraîchissement manuel.

3. Justification technique

React Native et Expo

Le projet a été réalisé avec React Native et Expo car il s'agit de la stack utilisée dans le cours. Ce choix permet de rester cohérent avec la matière vue et de faciliter les tests sur smartphone ou simulateur.

TypeScript

TypeScript a été utilisé pour garder un typage clair dans les composants, les formulaires, les hooks et les données SQLite. Cela aide à limiter les erreurs et à rendre le code plus lisible.

Expo Router

Expo Router a été utilisé pour la navigation car c'est la solution vue au cours. La navigation par fichiers permet une structure simple avec :

- un layout racine ;
- deux écrans principaux dans les onglets ;
- un écran dynamique de détail d'une séance.

SQLite

SQLite a été choisi car les données du projet sont relationnelles :

- une séance contient plusieurs exercices ;
- un exercice contient plusieurs séries.

Le stockage local est suffisant pour ce sujet et évite d'ajouter un serveur ou une API. Le projet utilise SQLiteProvider, useSQLiteContext, des migrations avec PRAGMA user_version, l'activation de foreign_keys et le mode WAL.

React Hook Form et Zod

React Hook Form et Zod ont été choisis pour rester conformes au cours et centraliser la validation des formulaires. React Hook Form gère l'état des formulaires et la soumission, tandis que Zod gère les règles de validation et les messages d'erreur.

Gestion d'état

La gestion d'état a été volontairement gardée simple avec :

- useState ;
- useEffect ;
- useRef ;
- des hooks personnalisés pour l'accès aux données.

Il n'y a pas de bibliothèque externe de state management, car ce n'était pas nécessaire pour un projet de cette taille.

Haptics

La bibliothèque expo-haptics est utilisée pour confirmer visuellement et tactilement la complétion d'une série. Cela améliore légèrement l'expérience utilisateur sans complexifier l'application.

Objectif hebdomadaire

L'objectif hebdomadaire a été implémenté dans sa version la plus simple pour rester cohérent avec le niveau attendu du projet. L'objectif est fixé à trois séances par semaine et la progression est calculée à partir des dates déjà présentes dans la base SQLite.

Ce choix évite d'ajouter une configuration supplémentaire, tout en répondant à la fonctionnalité demandée dans le sujet.

4. Architecture

Structure des dossiers

- app/ : routes Expo Router ;
- app/(tabs)/ : écrans Aujourd'hui et Archives ;
- app/session/[id].tsx : détail d'une séance ;
- src/components/ : composants réutilisables ;
- src/constants/ : constantes visuelles ;
- src/context/ : provider SQLite ;
- src/db/ : migrations et types liés à la base de données ;
- src/hooks/ : logique d'accès aux données et mutations.

Cette séparation permet de ne pas mélanger la navigation, l'interface et l'accès à la base de données dans les mêmes fichiers.

Flux de données

Création d'une séance

1. L'utilisateur complète le formulaire sur l'écran Aujourd'hui.
2. React Hook Form récupère les valeurs.
3. Zod valide les données.
4. Le hook de création enregistre la séance et les exercices dans SQLite.
5. Une transaction garantit la cohérence de l'insertion.
6. L'utilisateur est redirigé vers le détail de la séance.

Détail d'une séance

1. L'identifiant de la séance est récupéré depuis la route.
2. Un hook charge la séance, ses exercices et ses séries.
3. L'utilisateur peut ajouter un exercice ou des séries.

4. Les modifications passent par un hook de mutations SQLite.
5. L'écran recharge les données après modification.

Historique

1. Un hook charge les séances depuis SQLite.
2. Une requête SQL calcule les informations utiles à l'affichage.
3. Le hook calcule aussi le nombre de séances de la semaine en cours pour l'objectif hebdomadaire.
4. Les résultats sont affichés dans une FlatList.

5. Auto-évaluation

Le tableau suivant compare les fonctionnalités attendues dans le sujet et ce qui a été réalisé dans l'application.

Fonctionnalité attendue	Réalisée	Commentaire
Créer une séance avec date, durée et commentaire	Oui	Formulaire complet avec validation
Ajouter des exercices dans la séance	Oui	Ajout de plusieurs exercices
Ajouter des séries par exercice avec répétitions et charge	Oui	Poids et répétitions enregistrés dans SQLite
Historique des séances et détail d'une séance	Oui	Onglet Archives et écran de détail
Objectif hebdomadaire avec progression	Oui	Objectif fixe de 3 séances avec barre de progression

Bilan

Les cinq fonctionnalités principales demandées dans le sujet sont présentes dans l'application.

En complément du sujet de base, l'application propose aussi :

- la complétion des séries ;
- un retour haptique ;
- des statistiques simples ;
- la gestion des états de chargement et d'erreur.

Conclusion

MyWorkoutApp répond correctement à la majeure partie du sujet demandé. L'application permet de créer des séances, ajouter des exercices, enregistrer des séries et consulter l'historique de manière claire et locale.

Le projet reste simple dans son architecture et dans les technologies utilisées, tout en respectant les notions vues au cours. L'application est cohérente, stable et défendable dans le cadre d'un projet de

troisième année.